

REPASO PERSONAL

Guia de defensa con enfoque de inteligencia artificial

Documento de estudio para explicar Agora como un proyecto funcional desarrollado con apoyo de IA generativa, de forma honesta y defendible.

Indice

- 1. Enfoque principal que voy a defender
- 2. Que parte he hecho yo
- 3. Que papel ha tenido la IA en el desarrollo
- 4. Que quiero demostrar con esta defensa
- 5. Tecnologías usadas y como las explico
- 6. Como se conectan frontend y backend a nivel de código
- 7. Errores que entiendo y puedo explicar
- 8. Dificultades reales que tuve
- 9. Flujo funcional principal
- 10. Funcionalidades que puedo defender como cerradas
- 11. Funcionalidades que reconozco como futuras mejoras
- 12. Preguntas difíciles y respuestas
- 13. Mini guion para decirlo oralmente

Guia de defensa: como voy a explicar Agora con IA y tecnologías actuales

1. Enfoque principal que voy a defender

No tengo que cambiar el proyecto ni presentarlo como otro proyecto distinto. Lo que tengo que cambiar es el enfoque de la defensa.

Agora sigue siendo una aplicación funcional para gestionar prácticas entre un centro educativo y empresas colaboradoras. Lo que voy a explicar es como, con las tecnologías actuales y con apoyo de inteligencia artificial, he podido desarrollar, integrar, desplegar y validar un sistema completo.

La defensa no tiene que sonar a:

He programado manualmente todo este sistema desde cero.

Tiene que sonar a:

He desarrollado un proyecto funcional usando herramientas actuales de desarrollo, despliegue e inteligencia artificial. El objetivo es demostrar como se puede construir una aplicacion real combinando frontend, backend, base de datos, cloud, Docker, correo, pruebas y supervision tecnica.

Para mi, el valor del proyecto no esta solo en el codigo generado, sino en haber conseguido convertir una idea en un sistema que se puede abrir desde una URL, que tiene portal interno, portal externo, base de datos, correo, despliegue en una VM y una herramienta de escritorio para supervision.

Si me preguntan directamente si he programado todo yo, mi respuesta tiene que ser clara:

No he escrito manualmente todo el codigo linea por linea. He usado IA generativa y herramientas actuales como apoyo para construir el sistema. Mi trabajo ha sido orientar el desarrollo, definir requisitos, probar flujos, revisar errores, adaptar el despliegue y entender como encajan las tecnologias. Por eso la defensa se centra en explicar el proceso, las tecnologias utilizadas y el resultado funcional.

La idea clave es:

No intento convencer de que todo el codigo es mio de forma manual, sino demostrar que se usar las herramientas actuales para llevar una aplicacion desde la idea hasta un despliegue funcional.

2. Que parte he hecho yo

Mi aportacion se puede explicar en varias partes.

Primero, he definido el problema. El sistema intenta resolver la gestion de practicas entre un centro educativo y empresas colaboradoras. Eso incluye empresas, estudiantes, tutores, convenios, asignaciones, seguimiento, documentos y comunicacion.

Despues, he ido definiendo que necesitaba cada portal. El portal interno tenia que servir para que el centro gestionara empresas, convenios, estudiantes y asignaciones. El portal externo tenia que permitir que una empresa se registrara, verificara su correo, enviara una solicitud y pudiera comunicarse con el centro.

Tambien he trabajado en la validacion funcional. He probado inicios de sesion, errores 401 y 500, aprobacion y rechazo de empresas, mensajeria, asignaciones, tutores, documentos, URL publica, despliegue cloud y arranque automatico.

Ademas, he tenido que tomar decisiones de alcance. Algunas funcionalidades eran demasiado grandes para cerrarlas bien, como la gestion documental avanzada, roles muy detallados o multi-

centro. En esos casos las he dejado como mejoras futuras en vez de intentar defender que estaban completas.

Una forma sencilla de explicarlo seria:

La IA y las herramientas actuales me han ayudado mucho a avanzar, pero yo he tenido que dirigir que se queria construir, probar si funcionaba, detectar incoherencias y decidir que se quedaba dentro del alcance final.

3. Que papel ha tenido la IA en el desarrollo

La IA ha tenido un papel importante como herramienta de apoyo. No la presento como sustituta del trabajo, sino como una tecnologia actual que permite acelerar la creacion de una aplicacion si se sabe guiar, probar y corregir.

Me ha ayudado a generar codigo de backend, frontend, scripts de despliegue, pruebas, documentacion y correcciones.

En concreto, la IA ha ayudado en:

- estructura de controladores Symfony;
- componentes y formularios React;
- endpoints de API;
- validaciones de datos;
- mensajes de error;
- pruebas automatizadas;
- scripts de Docker y despliegue;
- documentacion tecnica;
- guiones de presentacion.

Pero el aprendizaje importante es que la IA no hace que todo funcione automaticamente. Muchas veces genera algo que parece correcto, pero luego falla al probarlo en un entorno real.

Por ejemplo, hubo migraciones que parecian validas, pero fallaban en PostgreSQL. Tambien hubo errores de permisos en Symfony, problemas con la URL publica, problemas de documentos y errores que aparecian directamente al usuario.

Por eso puedo decir:

La IA acelera el desarrollo, pero no sustituye la revision. Hay que probar, mirar logs, entender los errores y corregir el sistema hasta que el flujo sea demostrable. Ese es precisamente uno de los aprendizajes principales del proyecto.

4. Que quiero demostrar con esta defensa

Con esta defensa quiero demostrar cuatro cosas.

Primero, que he entendido el problema funcional: un centro necesita gestionar empresas, convenios, estudiantes, asignaciones y comunicacion con empresas.

Segundo, que he sabido usar tecnologias actuales para construir una solucion:

- React para los portales;
- Symfony para el backend;
- Doctrine para conectar con la base de datos;
- PostgreSQL para persistencia;
- Docker y Docker Compose para despliegue;
- Google Cloud para publicar el sistema;
- Caddy/HTTPS para exponer la URL;
- Brevo para correo;
- Electron para Agora Desktop;
- IA generativa como apoyo al desarrollo.

Tercero, que he aprendido a integrar esas piezas. El proyecto no es solo una pantalla aislada: hay frontend, backend, base de datos, contenedores, despliegue, correo, logs y pruebas.

Cuarto, que he aprendido de los fallos. Durante el proyecto aparecieron errores reales y eso me obligo a revisar logs, migraciones, permisos, rutas, codigos de error y persistencia.

Frase para decirlo:

El objetivo de mi defensa no es decir que he escrito manualmente cada linea, sino explicar como he usado un conjunto de tecnologias actuales, incluida la IA, para construir y desplegar una aplicacion funcional.

5. Tecnologias usadas y como las explico

Symfony

Symfony es el backend principal. Es la parte que recibe las peticiones del frontend, valida los datos, aplica la logica de negocio y guarda o consulta informacion en la base de datos.

En el proyecto, Symfony se usa para crear la API. Por ejemplo, hay rutas como:

- `/api/empresas`
- `/api/convenios`

- `/api/asignaciones`
- `/api/monitor`
- `/api/portal-company/overview`

Cuando el frontend necesita datos, no habla directamente con PostgreSQL. Llama a una ruta de Symfony, y Symfony responde con JSON.

Frase para decirlo:

Symfony es el nucleo del backend. Recibe las peticiones de los portales, valida los datos, usa Doctrine para acceder a la base de datos y devuelve respuestas JSON.

React

React es la tecnologia usada para construir las interfaces de usuario.

Hay dos frontends principales:

- portal interno del centro: `frontend/app`;
- portal externo de empresas: `frontend/company-portal`.

React se encarga de mostrar pantallas, formularios, tablas, botones y mensajes. Cuando el usuario hace una accion, React llama a la API del backend.

Frase:

React es la capa visual. Muestra los datos y recoge las acciones del usuario, pero la logica principal y la persistencia estan en el backend.

Doctrine

Doctrine es el ORM que conecta Symfony con la base de datos.

En vez de escribir SQL manual para todo, se trabaja con clases PHP llamadas entidades. Por ejemplo:

- `EmpresaColaboradora`
- `Convenio`
- `AsignacionPractica`
- `Estudiante`
- `TutorAcademico`
- `TutorProfesional`

Doctrine se encarga de traducir esas entidades a tablas y consultas SQL.

Frase:

Doctrine me permite trabajar con objetos del dominio, como empresas o convenios, y el ORM se encarga de llevar esos datos a PostgreSQL.

PostgreSQL

PostgreSQL es la base de datos que se usa en el despliegue cloud.

Guarda la informacion persistente del proyecto:

- empresas;
- estudiantes;
- tutores;
- convenios;
- asignaciones;
- mensajes;
- cuentas de empresa;
- solicitudes;
- auditoria.

Frase:

PostgreSQL es la base de datos real del despliegue. Es mas adecuada que SQLite para un entorno cloud porque esta pensada para uso persistente y concurrente.

Docker y Docker Compose

Docker sirve para empaquetar la aplicacion en contenedores. En vez de instalar manualmente PHP, Apache, PostgreSQL y Caddy en la VM, cada parte se ejecuta en un contenedor.

Docker Compose define como se levantan juntos esos contenedores.

En la VM se usan principalmente:

- contenedor de aplicacion: Symfony, Apache y los builds de React;
- contenedor de base de datos: PostgreSQL;
- contenedor proxy: Caddy para exponer HTTP/HTTPS.

Frase:

Docker hace que el despliegue sea mas reproducible. Docker Compose define los servicios, redes, volúmenes y variables necesarias para levantar todo el sistema.

Google Cloud VM

La VM de Google Cloud es el servidor donde se ejecuta el proyecto.

Sirve para que la aplicacion no funcione solo en mi ordenador, sino que este accesible desde Internet mediante una URL publica.

Frase:

La VM permite demostrar el proyecto en un entorno parecido a produccion, con Docker, PostgreSQL, HTTPS y acceso externo.

Caddy y HTTPS

Caddy actua como proxy. Recibe las peticiones externas por los puertos 80 y 443 y las redirige al contenedor de la aplicacion.

Tambien ayuda a servir la aplicacion por HTTPS.

Frase:

Caddy esta delante de la aplicacion y se encarga de exponerla hacia fuera de forma mas limpia, usando HTTP y HTTPS.

Brevo

Brevo se usa para el envio de correos.

En el proyecto se usa sobre todo para:

- correo de verificacion de cuenta o solicitud;
- notificaciones relacionadas con solicitudes cuando el correo esta configurado.

Frase:

Brevo es el servicio externo de correo. Lo uso para que el portal externo pueda enviar verificaciones y notificaciones reales.

Agora Desktop

Agora Desktop es una aplicacion de escritorio hecha con Electron.

No es el portal principal. Es una herramienta tecnica para:

- ver estado del despliegue;
- comprobar URL cloud;
- consultar logs;
- lanzar validaciones;

- trabajar en modo cloud o local.

Frase:

Agora Desktop me sirve como consola tecnica. Me permite comprobar rapidamente si el cloud responde, ver logs y tener una forma de supervision sin depender solo del navegador.

6. Como se conectan frontend y backend a nivel de codigo

Esta es una parte importante porque si me preguntan que significa "conectar frontend y backend", no puedo quedarme en algo generico.

La explicacion base es:

El frontend no accede directamente a la base de datos. El usuario interactua con React. React llama a una API usando `fetch`. Esa API esta hecha en Symfony. Symfony valida la peticion, consulta o guarda datos con Doctrine y devuelve JSON. React recibe ese JSON y actualiza la pantalla.

Archivos que puedo mencionar

Portal interno:

- `frontend/app/src/App.tsx` : organiza la aplicacion interna, las vistas y el estado principal.
- `frontend/app/src/services/api.ts` : centraliza las llamadas HTTP al backend.
- `frontend/app/src/types.ts` : define tipos de datos como empresa, convenio, estudiante o asignacion.

Portal externo:

- `frontend/company-portal/src/App.tsx` : gestiona registro, login de empresa, estado, panel externo y mensajes.

Backend:

- `backend/src/Controller/Api/EmpresaColaboradoraController.php` : endpoints de empresas.
- `backend/src/Controller/Api/ConvenioController.php` : endpoints de convenios.
- `backend/src/Controller/Api/AsignacionController.php` : endpoints de asignaciones y seguimiento.
- `backend/src/Controller/Api/PortalCompanyController.php` : endpoints del portal externo.
- `backend/src/Controller/PortalAuthController.php` : registro y login del portal externo.
- `backend/src/Controller/RegistroEmpresaController.php` : solicitud publica y verificacion de correo.

Entidades:

- `backend/src/Entity/EmpresaColaboradora.php`
- `backend/src/Entity/EmpresaSolicitud.php`
- `backend/src/Entity/EmpresaPortalCuenta.php`
- `backend/src/Entity/Convenio.php`
- `backend/src/Entity/AsignacionPractica.php`
- `backend/src/Entity/EmpresaMensaje.php`

Ejemplo 1: cargar empresas

Cuando en el portal interno se muestra la lista de empresas, el recorrido es:

1. React necesita cargar empresas.
2. El cliente de API del frontend llama a `GET /api/empresas`.
3. Esa llamada esta centralizada en `frontend/app/src/services/api.ts`.
4. Symfony recibe la peticion en `EmpresaColaboradoraController.php`.
5. El controlador usa el repositorio de empresas.
6. Doctrine consulta PostgreSQL.
7. Symfony devuelve un JSON con las empresas.
8. React pinta la tabla.

Como lo diria:

Por ejemplo, para cargar empresas, React llama a `/api/empresas`. Esa ruta la atiende Symfony en el controlador de empresas. El controlador consulta las entidades con Doctrine y devuelve JSON. Despues React usa ese JSON para pintar la tabla.

Ejemplo 2: crear una empresa

Cuando se crea una empresa desde el portal interno:

1. El usuario rellena el formulario.
2. React crea un objeto con los datos.
3. El frontend envia un `POST /api/empresas`.
4. Symfony recibe el payload.
5. El backend valida campos como nombre, email, estado o sector.
6. Si hay errores, devuelve un mensaje controlado.
7. Si todo esta bien, crea una entidad `EmpresaColaboradora`.
8. Doctrine la guarda en PostgreSQL con `persist` y `flush`.
9. El backend devuelve la empresa creada.

10. React actualiza la vista.

Palabras que puedo explicar:

- `POST` : peticion para crear datos.
- `payload` : datos que se envian desde el formulario.
- `validacion` : comprobacion de que los datos tienen sentido.
- `EntityManager` : componente que gestiona guardado de entidades.
- `persist` : preparar entidad para guardar.
- `flush` : ejecutar los cambios en base de datos.

Frase:

Crear una empresa es un flujo completo: formulario en React, peticion POST a Symfony, validacion en backend, entidad Doctrine y respuesta JSON para refrescar la pantalla.

Ejemplo 3: registro de empresa externa

En el portal externo el flujo es distinto porque lo inicia una empresa desde fuera del centro.

1. La empresa entra en `/externo`.
2. Crea una cuenta con email y contrasena.
3. Despues rellena los datos de empresa: nombre, CIF/NIF, sector, contacto y tutor profesional.
4. El portal externo envia la solicitud al backend.
5. Symfony crea una `EmpresaSolicitud`.
6. Se genera un token de verificacion.
7. Se envia correo con Brevo si esta configurado.
8. El centro revisa la solicitud desde el portal interno.
9. Si se aprueba, la empresa pasa a `EmpresaColaboradora` activa.

Como lo diria:

En el portal externo separo la cuenta de empresa y la solicitud. Primero la empresa puede tener acceso al portal, pero hasta que el centro aprueba la solicitud no se convierte en empresa colaboradora activa.

Ejemplo 4: crear convenio y asignacion

Para crear un convenio:

1. El centro selecciona una empresa activa.
2. React envia `POST /api/convenios`.
3. Symfony valida que la empresa exista y que los datos del convenio sean validos.

4. Doctrine guarda la entidad `Convenio`.

Para crear una asignacion:

1. Se selecciona estudiante, empresa, convenio, tutor academico y fechas.

2. React envia `POST /api/asignaciones`.

3. Symfony valida que:

- el estudiante exista; - la empresa este activa; - el convenio sea valido; - el tutor academico exista; - el estudiante no tenga otra practica activa incompatible.

1. Si todo esta bien, se crea `AsignacionPractica`.

Frase:

La asignacion tiene mas logica que un simple formulario, porque el backend tiene que comprobar reglas del dominio: empresa activa, convenio valido y que el estudiante no tenga conflictos.

Ejemplo 5: mensajes empresa-centro

La mensajeria se guarda en la entidad `EmpresaMensaje`.

El flujo es:

1. La empresa escribe desde el portal externo.

2. El backend guarda el mensaje con autor `empresa`.

3. El portal interno puede ver el hilo.

4. El centro responde.

5. El mensaje se guarda con autor `centro`.

6. El portal externo consulta periodicamente la API para refrescar mensajes.

Frase:

No es un chat con WebSockets en tiempo real. Es una mensajeria persistida en base de datos con refresco automatico desde el frontend.

7. Errores que entiendo y puedo explicar

Durante el desarrollo aparecieron varios tipos de errores. Es importante saber explicarlos.

Error 401

Significa que el usuario no esta autenticado.

Ejemplo:

Intento acceder a una ruta privada sin iniciar sesion o con credenciales incorrectas.

Error 403

Significa que el usuario esta autenticado, pero no tiene permisos.

Ejemplo:

Un profesor intenta borrar una empresa, pero esa accion esta limitada al usuario admin.

Error 404

Significa que el recurso no existe.

Ejemplo:

Se intenta abrir un convenio o empresa con un ID que no existe en la base de datos.

Error 422

Significa que los datos enviados no pasan la validacion.

Ejemplo:

Falta un CIF/NIF obligatorio o un campo no tiene el formato esperado.

Error 500

Significa error interno del servidor.

Ejemplos reales que aparecieron:

- problemas de permisos de cache en Symfony;
- migraciones incorrectas;
- errores de base de datos;
- excepciones no controladas.

Frase:

Una parte del trabajo fue que estos errores no aparecieran de forma cruda al usuario. El backend intenta devolver mensajes controlados y el frontend los convierte en textos mas comprensibles.

8. Dificultades reales que tuve

Migraciones entre SQLite y PostgreSQL

Una dificultad importante fue que algunas migraciones funcionaban en SQLite, pero fallaban al desplegar con PostgreSQL.

Por ejemplo, aparecieron errores con `AUTOINCREMENT` y `CLOB`, que son sintaxis que no encajaban bien con PostgreSQL.

Como lo explico:

El problema fue que el entorno local y el entorno cloud no se comportaban igual. Tuve que revisar logs, entender que la migracion estaba generada con sintaxis no compatible y adaptarla para que PostgreSQL pudiera crear las tablas.

Permisos de Symfony

Tambien aparecieron errores 500 porque Symfony no podia escribir en `var/cache/prod`.

Como lo explico:

La aplicacion arrancaba, pero al entrar en la API fallaba porque Symfony necesitaba escribir archivos de cache dentro del contenedor. Se corrigio ajustando permisos para que Apache/PHP pudiera escribir donde correspondia.

IP publica y URL

Al reiniciar la VM, la IP publica podia cambiar.

Para solucionarlo se uso `nip.io`, que permite construir una URL usando la IP.

Como lo explico:

Como no tenia un dominio propio definitivo, use una URL con nip.io. Asi podia acceder con una direccion tipo `agora.IP.nip.io`. Tambien se adapto el despliegue para poder recalculat la URL si cambia la IP.

Documentos

La parte documental fue una de las mas problematicas.

Habia que guardar dos cosas:

- metadatos en base de datos;
- archivo real en almacenamiento persistente.

Como lo explico:

La subida de documentos no era solo un formulario. Habia que asegurar que el archivo se guardara en un volumen persistente y que la base de datos mantuviera la referencia correcta. Por eso la deje como una parte funcional en algunos casos, pero tambien como mejora futura para hacerla mas robusta.

Alcance demasiado grande

El proyecto crecio mucho.

Acabo incluyendo:

- portal interno;
- portal externo;
- backend;
- base de datos;
- correo;
- chat;
- documentos;
- cloud;
- desktop;
- roles;
- memoria;
- presentacion.

Como lo explico:

Una de las mayores dificultades fue controlar el alcance. Con IA se avanza rapido, pero tambien es facil abrir demasiadas funcionalidades. Aprendi que hay que priorizar el flujo principal y dejar el resto como mejoras futuras.

9. Flujo funcional principal

Flujo de empresa

El flujo empieza en el portal externo.

1. La empresa crea una cuenta.
2. Rellena los datos de empresa.
3. Incluye CIF/NIF y tutor profesional.

4. Verifica el correo.
5. El centro revisa la solicitud.
6. Si se aprueba, la empresa queda activa.
7. Si se rechaza, queda registrado el motivo.

Frase:

Este flujo representa la entrada de una empresa nueva al sistema. No se crea directamente como colaboradora activa, sino que pasa por una solicitud y una revision del centro.

Flujo de convenio

Despues de aprobar una empresa, el centro puede crear un convenio.

1. Se selecciona la empresa.
2. Se definen fechas y condiciones.
3. Se indica el estado del convenio.
4. Se puede asociar documentacion.

Frase:

El convenio representa el acuerdo entre el centro y la empresa. Sin una empresa activa y un convenio valido, no tiene sentido crear una asignacion de practicas.

Flujo de asignacion

La asignacion une las piezas principales:

- estudiante;
- empresa;
- convenio;
- tutor academico;
- tutor profesional;
- fechas;
- modalidad;
- horas;
- estado.

Frase:

La asignacion es el punto donde se concreta la practica. Une al estudiante con una empresa y un convenio, y permite hacer seguimiento y evaluacion.

10. Funcionalidades que puedo defender como cerradas

Puedo defender como funcional:

- portal interno accesible en cloud;
- portal externo accesible en cloud;
- registro de empresa;
- verificacion de correo;
- solicitud de colaboracion;
- aprobacion y rechazo desde el centro;
- gestion de empresas;
- gestion de estudiantes;
- gestion de tutores;
- gestion de convenios;
- gestion de asignaciones;
- mensajeria empresa-centro;
- datos realistas de demostracion;
- despliegue en VM con Docker;
- arranque automatico con `agora.service`;
- supervision desde Agora Desktop.

11. Funcionalidades que reconozco como futuras mejoras

Estas partes no las debo vender como cerradas al 100 %:

- gestion documental mas robusta;
- roles avanzados y permisos finos;
- perfiles solo lectura;
- multi-centro;
- dominio propio estable;
- backups automaticos;
- mas pruebas E2E;
- auditoria mas completa;
- seguridad avanzada para produccion.

Como lo explico:

He priorizado el flujo principal. Algunas funciones estan planteadas o parcialmente trabajadas, pero para llevarlas a nivel de produccion necesitarian mas tiempo, por eso las presento como mejoras futuras.

12. Preguntas dificiles y respuestas

"Has hecho tu todo el codigo?"

No linea por linea. He usado IA generativa y tecnologias actuales como apoyo para construir el proyecto. Mi papel ha sido orientar el desarrollo, definir requisitos, probar flujos, revisar errores, validar el despliegue y documentar el resultado. Lo definiendo como un proyecto funcional desarrollado con apoyo de herramientas actuales, no como una autoria manual completa de cada linea.

"Entonces que has aprendido?"

He aprendido a entender como se estructura una aplicacion completa: React en el frontend, Symfony en el backend, Doctrine como conexion con la base de datos, PostgreSQL para persistencia y Docker para desplegarlo. Tambien he aprendido a seguir el recorrido de una accion desde que el usuario pulsa un boton hasta que se guarda en la base de datos.

"Entonces cual es el valor del proyecto si has usado IA?"

El valor esta en saber usar la IA como herramienta dentro de un proceso completo. No basta con pedir codigo. Hay que definir el problema, probar lo generado, detectar errores, integrar tecnologias, desplegarlo en cloud y explicar que funciona y que queda como mejora futura.

"Explica como se conecta frontend y backend"

El usuario interactua con React. React llama a una ruta de la API, por ejemplo `/api/empresas`. Symfony recibe esa peticion en un controlador, valida los datos, usa Doctrine para consultar o guardar en PostgreSQL y devuelve JSON. React recibe ese JSON y actualiza la interfaz.

"Que es un controlador?"

Es la clase de Symfony que recibe una peticion HTTP. Por ejemplo, `EmpresaColaboradoraController` atiende rutas de empresas. El controlador valida la peticion, coordina la logica y devuelve una respuesta.

"Que es una entidad?"

Es una clase PHP que representa un concepto del dominio y se relaciona con una tabla de la base de datos. Por ejemplo, `EmpresaColaboradora` representa empresas y `AsignacionPractica` representa una practica asignada a un estudiante.

"Que es una API?"

Es la capa de comunicacion entre el frontend y el backend. En mi proyecto son rutas como `/api/empresas`, `/api/convenios` o `/api/asignaciones`, que devuelven datos en JSON.

"Que es JSON?"

Es el formato de datos que viaja entre React y Symfony. Por ejemplo, una empresa se puede enviar como un objeto con nombre, sector, email y estado.

"Que dificultades tecnicas tuviste?"

Las principales fueron migraciones incompatibles entre SQLite y PostgreSQL, permisos de cache en Symfony dentro de Docker, errores 401 y 500, gestion de documentos, URL publica cambiante y controlar un alcance demasiado amplio.

"Por que hay partes como futuras mejoras?"

Porque el proyecto abarco mucho. Preferi cerrar bien el flujo principal y reconocer como mejoras futuras las partes que necesitaban mas tiempo para ser robustas, como documentos avanzados, multi-centro o roles finos.

"Que harias distinto?"

Cerraria antes el alcance, usaria desde el principio una base de datos igual en local y cloud, escribiria pruebas antes de ampliar funcionalidades y documentaria mejor cada decision tomada con IA.

"Que riesgo tiene usar IA para programar?"

Que puede generar codigo que parece correcto, pero falla en entorno real. Por eso hay que probar, revisar logs, validar con datos reales y no aceptar el resultado sin entender el flujo.

13. Mini guion para decirlo oralmente

Mi proyecto se llama Agora y se centra en la gestion de practicas entre un centro educativo y empresas colaboradoras.

El enfoque que voy a defender es como he podido desarrollar y desplegar una aplicacion funcional usando tecnologias actuales, incluida la inteligencia artificial como apoyo al desarrollo. No quiero presentarlo como si hubiera escrito manualmente cada linea de codigo, sino como un proyecto donde he usado herramientas modernas para llevar una idea a un sistema real.

La aplicacion tiene un portal interno para el centro, un portal externo para empresas, un backend con Symfony, una base de datos PostgreSQL, despliegue en Google Cloud con Docker y una aplicacion de escritorio para supervision tecnica.

A nivel tecnico, el frontend y el backend se comunican mediante una API. Por ejemplo, cuando el usuario consulta empresas, React llama a `/api/empresas`; Symfony recibe esa peticion en un controlador, usa Doctrine para consultar PostgreSQL y devuelve JSON para que React actualice la pantalla.

El flujo principal empieza cuando una empresa se registra en el portal externo, verifica su correo y envia una solicitud. Desde el portal interno, el centro puede aprobarla o rechazarla. Si se aprueba, la empresa queda activa y se pueden crear convenios. Despues se pueden crear asignaciones de estudiantes, con tutores y seguimiento.

Durante el desarrollo aparecieron dificultades reales: migraciones que fallaban en PostgreSQL, permisos de cache en Symfony, errores 401 y 500, problemas con la URL publica y la gestion documental. Esto me sirvio para aprender que la IA y las herramientas actuales ayudan mucho, pero no sustituyen la prueba, la revision tecnica ni la comprension del flujo.

La conclusion es que Agora demuestra como se puede usar IA generativa junto con tecnologias web actuales para construir una aplicacion completa y desplegada. Tambien muestra sus limites: hay que entender el flujo, probarlo y ser honesto con que partes estan cerradas y que partes quedan como mejoras futuras.